

DTSC 2302 Final Project

Esteban Grijalva, Emmett Lashbrook, Brandon Miller, Yager Range,
Peyton Robinson

Introduction

From all the potential datasets available we decided on two we found as interesting. A dataset recording various car related incidents and a dataset recording population change in the city of Charlotte. We were curious as to how these two sets were potentially related, whether immigration and increase in population caused a linear increase in the crashes or maybe it was a more exponential increase. For regression, we wanted to model population change vs the number of traffic accidents. On the classification side we figured we could take this data and use it to find out high versus low traffic fatality risk years in Mecklenburg County. In our process we made several models for both regression and classification. We worked to modify features as well as types of modeling to get the best possible predictions. Below is our process for each of the types of model we created.

Model 1 Regression

This regression analysis was built to answer: how strongly do population change and recent crash history help explain annual traffic accidents in Mecklenburg County, the Charlotte proxy used for interpretation in the "County Estimate" data. To make the model strong, it was built not to rely on a single predictor or a single fitting method.

The final feature set was chosen to capture both scale and trend. Population level measures the size of the county at a given time, while population change and population growth percentage describe how quickly the county is expanding. Crash lags and crash-growth lags

provide the recent accident history that a yearly transportation series would depend on. A year trend variable was also included so the models could learn long run movement in crashes over time, and a three year rolling population average was added to smooth population fluctuations. Crashes may increase in an exponential or nonlinear way, the response variable was modeled on a log scale inside the comparison models. That choice makes the linear model more realistic for count growth and gives the spline based model room to bend rather than forcing a perfectly straight relationship which can be unrealistic.

To address multicollinearity observed in initial diagnostics (very high VIFs for `year\_trend`, `log\_population`, and `population\_roll3`), I applied an VIF-pruning step that removed the highest-VIF predictor until all remaining features had acceptable VIFs (or until five features remained). This reduces instability in coefficient estimates while keeping an interpretable set of predictors.

> Notes: VIFs remained above 10 for a few features because the pruning stopped at five features to preserve interpretability; these remaining VIFs are reported above and should be considered when interpreting coefficient magnitudes.

Linear Regression achieved the lowest error with an RMSE of 9,897.65 crashes, outperforming the nonlinear methods (Random Forest: 10,192, KNN: 10,225, GAM splines(AI was used for this model): 10,821). Despite having higher cross validation error, Linear Regression generalized best to the held out test set. The standardized coefficients from the linear model show that (after pruning) prior year crashes and population related variables are meaningful predictors. This suggests the relationships between primary drivers are straightforward: crashes scale with recent crash history and population movement.

Limits (SMALL SAMPLE & MULTICOLLINEARITY):

- The dataset is small (42 observations, 9 test rows). Small samples lead to high variance in metric estimates and coefficient instability; negative R^2 values and large coefficient swings across models can occur. Interpret results with caution and prioritize effect direction and relative importance rather than point estimates.
- Multicollinearity was observed and addressed via VIF-based pruning; remaining features have reduced VIFs, but correlated predictors can still inflate standard errors.

Simpler models sometimes generalize better than complex nonlinear approaches, especially with limited sample size. The linear model's better performance does not mean crashes grow perfectly linearly with population. Rather, it suggests that among the features included, a linear model captures the test set patterns most accurately. The log-transformed target ensures the model is appropriate for count based data where crash growth may accelerate, while the inclusion of lagged crashes and population trends provides the context needed for good predictions.

Model 2 Classification

Our goal when we started making the classification model was to develop a binary classification model to predict high versus low traffic fatality risk years in Mecklenburg County, North Carolina. We used two datasets to create our model. The first was transportation data spanning from 1970-2024, containing features like alcohol-related accidents, speed violations, motorcyclist accidents, and pedestrian accidents. The second was population data in Mecklenburg from the same time period. This lets us factor the importance of different types of accidents and the effect of population counts on raw accident counts.

After gathering the sources, we decided on the variables to use. The first thing we did was set the target, classifying years as ‘high risk’ or ‘low risk’. Years were labeled using the median of Traffic Accident Fatalities across all years in the dataset (1981–2024). Years above the median were labeled 1 (high risk) and years at or below were labeled 0 (low risk), producing a near-perfectly balanced dataset of 22 high-risk and 21 low-risk years. Next was the features. First, we excluded traffic accident fatalities as it was the variable that was used to create the median threshold, and using it would have caused target leakage in the model. Next, Pedestrian Accidents and Pedestrians Injured were excluded as they had only seven years worth of data, which would have greatly lowered the dataset. This left us with the final five features we used: Alcohol Related Accidents per capita, Accidents Caused by Exceeding Legal Speed per capita, Motorcyclists Injured per capita, Bicyclists Injured per capita, and Population Growth Percentage.

Our next step was determining which classification method to use. Three classification methods were evaluated. All models were assessed using 5-fold Stratified Cross-Validation, and performance was measured across three metrics: Accuracy, F1 Score, and ROC-AUC. We used these to capture both overall correctness and the balance between precision and recall. Of the three models we tested, Logistic Regression was the least accurate with an accuracy of 0.608, an F1 score of 0.604, and a ROC-AUC score 0.615. The low scores were attributed to its assumption of linear relationships between predictors and the outcome, which was not true of our data. The second method was K-Nearest Neighbors, with a chosen size of $K=7$. KNN achieved the highest accuracy and ROC-AUC of the three models, confirming that the relationship between these behavioral features and fatality risk is non-linear in nature. Its final scores were an accuracy of 0.697, a F1 score of 0.617, and a ROC-AUC score 0.700. The method we ultimately

selected was a random forest model. It had the second best scores, with an accuracy of 0.678, a F1 score of 0.672, and a ROC-AUC score 0.675. The similarity between scores combined with random forest models having more interpretable features is why we chose it.

These were the Gini importance values for our predictors: Accidents Caused by Exceeding Legal Speed per capita was the most important feature in the Random Forest model with a Gini importance of 0.26. Alcohol Related Accidents per capita was the second most important feature with a Gini importance of 0.24. Motorcyclists Injured per capita ranked third with a Gini importance of 0.20. Bicyclists Injured per capita contributed moderately with a Gini importance of 0.16. Population Growth Percentage was the weakest predictor with a Gini importance of 0.12. The main conclusion was that more accidents that were exceeding speed limits and alcohol related correlated to more deadly years, while higher motorcyclist and bicyclist injuries would also contribute. Population growth had very little impact.

Findings and Conclusions

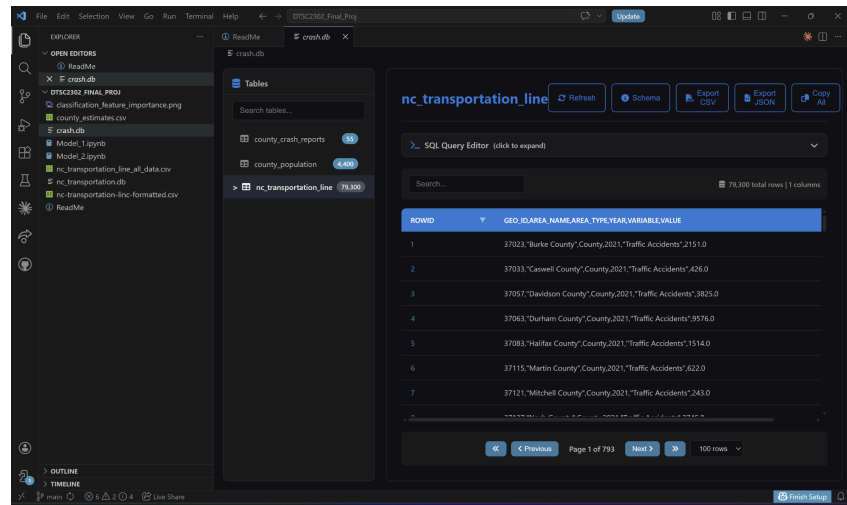
Across the various models we made in this project we found there to be some limitations in the datasets we chose. There were issues of small sample sizes within the regression model leading to a potentially less accurate model than what we had hoped for. Despite these limitations we managed to trim the dataset to eliminate as much collinearity as possible and managed to make a Linear Regression model that predicted decently given the data. The model's predictions deviated from actual crash counts by approximately 9,898 crashes on average. Given that Mecklenburg County reports tens of thousands of crashes annually, this error margin represents a reasonable but imperfect fit. It seems to have a decent prediction of the overall trend but is not fully accurate to the volatility of change in crashes year by year. The model worked but could use more time and data in the future to allow for better fine-tuning and accuracy.

The classification model provided its own challenges in creation leading to us dropping some data as it was a direct match to the target variable leading to a faulty model at first. Upon refining the model we were able to find importance values for each of the features we used. There were multiple conclusions to be drawn, the main one being accidents that included exceeding speed limits or were alcohol related correlated to more deadly years. However, it is important to note that despite logistic regression not being used, we did find that each coefficient for our targets was negatively correlated. This could mean that increased enforcement and reporting of speeding and alcohol incidents coincided with decades of declining fatality rates (potentially caused by increased safety measures in vehicles). Due to this, our main conclusion that was drawn was that the police correctly enforcing speeding and alcohol incidents combined with safety innovation will cause risk years to continue to drop. The model was successful, but things like adding the missing pedestrian incident data or adding the populations of other counties could help us make it more accurate.

Appendix

Link to [GitHub repository](#)

Screenshot 1 references the SQL database made by joining the csv files.



The screenshot shows a Jupyter Notebook interface with a dark theme. The left sidebar displays the file explorer with a project named 'DTSC2302_FINAL_Prog'. The main area shows a SQL query editor with the table 'nc_transportation_line' selected. The query result is displayed as a table with 7 rows and 1 column.

ROWID	GEO_ID,AREA,NAME,AREA,TYPE,YEAR,VARIABLE,VALUE
1	37023,"Burke County",County,2021,"Traffic Accidents",2151.0
2	37033,"Casswell County",County,2021,"Traffic Accidents",426.0
3	37057,"Davidson County",County,2021,"Traffic Accidents",3825.0
4	37063,"Durham County",County,2021,"Traffic Accidents",9576.0
5	37083,"Halifax County",County,2021,"Traffic Accidents",1514.0
6	37115,"Martin County",County,2021,"Traffic Accidents",622.0
7	37121,"Mitchell County",County,2021,"Traffic Accidents",243.0

(Screenshot 1)

Code Model 1

```
import sqlite3 as db
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.compose import ColumnTransformer, TransformedTargetRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score
from sklearn.model_selection import TimeSeriesSplit, cross_validate
from sklearn.neighbors import KNeighborsRegressor
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, SplineTransformer
from statsmodels.stats.outliers_influence import variance_inflation_factor
```

```
df = pd.read_csv('nc_transportation_line_all_data.csv', sep=';',
encoding='utf-16')

connection = db.connect('crash.db')
cursor = connection.cursor()

df.to_sql('nc_transportation_line', connection, if_exists='replace',
index=False)
connection.commit()
```

```
cursor.execute('SELECT * FROM nc_transportation_line')
rows = cursor.fetchall()
for row in rows[:5]:
    print(row)
print(f'Total rows in database: {len(rows)}')
```

```
# Load population data
pop = pd.read_csv('county_estimates.csv', sep=';')
pop.columns = pop.columns.str.replace('\ufeff', '',
regex=False).str.strip()
pop['FIPS'] = pop['FIPS'].astype(str).str.zfill(5)
pop['Year'] = pd.to_datetime(pop['Year'], errors='coerce').dt.year
```



```

pop = pop[(pop['Area Type'] == 'County') & (pop['Estimate'] == 'Annual
Estimate')][['FIPS', 'Area', 'Year', 'value']].rename(columns={'value':
'population'})

# Load crash data
linc = pd.read_csv('nc-transportation-linc-formatted.csv')
linc['Geo_ID'] = linc['Geo_ID'].astype(str).str.zfill(5)
linc_wide = linc.pivot_table(index=['Geo_ID', 'Year'], columns='Variable',
values='Value', aggfunc='first').reset_index().rename(columns={'Geo_ID':
'FIPS'})
linc_wide.columns.name = None

# Merge datasets
merged = pop.merge(linc_wide, on=['FIPS', 'Year'], how='inner')
merged = merged.sort_values(['FIPS', 'Year']).copy()

```

```

# SQL preview of the merged datasets
sql_connection = connection
pop.to_sql('county_population', sql_connection, if_exists='replace',
index=False)
linc_wide.to_sql('county_crash_reports', sql_connection,
if_exists='replace', index=False)

sql_merged = pd.read_sql_query(
    '''
    SELECT
        p.FIPS,
        p.Area,
        p.Year,
        p.population,
        c.\"Traffic Accidents\",
        c.\"Traffic Accident Fatalities\",
        c.\"Persons Injured in Traffic Accidents\",
        c.\"Alcohol Related Accidents\",
        c.\"Pedestrian Accidents\",
        c.\"Bicyclists Injured\",
        c.\"Motorcyclists Injured\",
        c.\"Accidents Caused by Exceeding Legal Speed\"
    FROM county_population AS p
    INNER JOIN county_crash_reports AS c

```

```

        ON p.FIPS = c.FIPS
        AND p.Year = c.Year
    ORDER BY p.FIPS, p.Year
    '''
    sql_connection
)

print('SQL merged dataset preview:')
print(sql_merged.head(28).to_string(index=False))
print(f'\nSQL merged dataset shape: {sql_merged.shape}')

```

```

# EDA: Quick correlation check and log transform for population
merged['log_population'] = np.log(merged['population'] + 1)
eda_cols = [c for c in ['population', 'log_population', 'Traffic
Accidents'] if c in merged.columns]
if len(eda_cols) > 1:
    corr = merged[eda_cols].corr(numeric_only=True)
    plt.figure(figsize=(6, 4))
    sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f')
    plt.title('Quick numeric correlation (population vs target)')
    plt.show()

# Feature engineering
df = merged.copy()
df['population_change'] = df.groupby('FIPS')['population'].diff()
df['population_growth_pct'] =
df.groupby('FIPS')['population'].pct_change() * 100
df['traffic_accidents_lag1'] = df.groupby('FIPS')['Traffic
Accidents'].shift(1)
df['traffic_accidents_growth_lag1'] = df.groupby('FIPS')['Traffic
Accidents'].transform(lambda series: series.pct_change().shift(1) * 100)
df['year_trend'] = df['Year'] - df['Year'].min()
df['population_roll13'] = df.groupby('FIPS')['population'].transform(lambda
series: series.rolling(3).mean())

# Feature set (at least 5 features per requirement)
target_col = 'Traffic Accidents'
feature_cols = [
    'log_population',
    'population_change',

```

```

    'population_growth_pct',
    'traffic_accidents_lag1',
    'traffic_accidents_growth_lag1',
    'year_trend',
    'population_roll13',
]
feature_cols = [col for col in feature_cols if col in df.columns]

model_df = df.dropna(subset=feature_cols + [target_col]).copy()

# VIF check for multicollinearity (from ModelBuildExample workflow
notebook)
def compute_vif(df_vif, features):
    X = df_vif[features].copy()
    X = X.fillna(0).assign(const=1)
    vif = pd.Series([variance_inflation_factor(X.values, i) for i in
range(X.shape[1]-1)], index=features)
    return vif

vif_series = compute_vif(model_df, feature_cols) if len(feature_cols) > 0
else pd.Series(dtype=float)
print('\nFeature set VIFs (higher than ~10 suggests multicollinearity):')
print(vif_series.round(2).to_string())

```

```

# VIF-based feature pruning to address multicollinearity
print('\n' + '='*80)
print('VIF-BASED FEATURE PRUNING')
print('='*80)

# Start with the originally selected features
_prune_features = feature_cols.copy()

#drop the feature with highest VIF until all VIFs <= 10 or until 5
features remain
while True:
    if len(_prune_features) <= 5:
        break
    vif_cur = compute_vif(model_df, _prune_features)
    max_vif = vif_cur.max()
    if max_vif > 10:

```

```

        drop_feat = vif_cur.idxmax()
        print(f'Dropping feature due to high VIF: {drop_feat}
(VIF={vif_cur[drop_feat]:.2f})')
        _prune_features.remove(drop_feat)
    else:
        break

feature_cols = _prune_features
model_df = df.dropna(subset=feature_cols + [target_col]).copy()

print('\nFinal feature set after pruning (kept at least 5 features):')
print(feature_cols)
print('\nFinal VIFs:')
print(compute_vif(model_df, feature_cols).round(2).to_string())
print('='*80)

```

```

# CORE PROJECT MODEL: 3-Feature Model
#
=====

==
print('\n')
print('=' * 80)
print('CORE 3-FEATURE MODEL')
print('=' * 80)
print()

# Use only the three core features specified in the project
core_features = ['population_change', 'population_growth_pct',
'traffic_accidents_growth_lag1']
core_df = model_df.dropna(subset=core_features + [target_col]).copy()

print(f'Using {len(core_features)} core features:')
for i, feat in enumerate(core_features, 1):
    print(f'  {i}. {feat}')
print()
print(f'Data points available: {len(core_df)}')
print()

# Split data (80/20 train-test)
core_cutoff = int(len(core_df) * 0.8)

```

```

core_train = core_df.iloc[:core_cutoff]
core_test = core_df.iloc[core_cutoff:]

X_core_train = core_train[core_features]
y_core_train = core_train[target_col]
X_core_test = core_test[core_features]
y_core_test = core_test[target_col]

# Fit linear model (easiest to interpret)
core_linear = LinearRegression()
core_linear.fit(X_core_train, y_core_train)

# Predictions
y_core_pred = core_linear.predict(X_core_test)
core_rmse = np.sqrt(mean_squared_error(y_core_test, y_core_pred))
core_mae = mean_absolute_error(y_core_test, y_core_pred)
core_r2 = r2_score(y_core_test, y_core_pred)

print('RESULTS: Linear Regression with 3 Features')
print('-' * 80)
print(f'Training set:  {len(core_train)} observations')
print(f'Test set:      {len(core_test)} observations')
print()
print('MODEL PERFORMANCE:')
print(f'  Root Mean Squared Error (RMSE): {core_rmse:.2f} crashes')
print(f'  Mean Absolute Error (MAE):      {core_mae:.2f} crashes')
print(f'  R2 Score:                      {core_r2:.4f}')
print()
print('FEATURE COEFFICIENTS (Impact on Crash Count):')
print('-' * 80)
for feat, coef in zip(core_features, core_linear.coef_):
    if 'change' in feat:
        print(f'{feat:40s}: {coef:8.4f} (crashes per person)')
    elif 'pct' in feat:
        print(f'{feat:40s}: {coef:8.2f} (crashes per 1%)')
    else:
        print(f'{feat:40s}: {coef:8.2f} (crashes per 1%)')

intercept = core_linear.intercept_
print()

```

```

print(f'Intercept (baseline crash count):
{intercept:8.2f}')
print()
print('ANSWER:')
print('-' * 80)
print(f'When population GROWS by 1%, crashes increase by approximately
{core_linear.coef_[1]:.2f}')
print(f'Relationship pattern: {"Linear increase" if core_linear.coef_[1] >
0 else "Linear decrease"}')
print('=' * 80)

```

```

# FULL MODEL vs CORE MODEL COMPARISON
print('\n' + '=' * 80)
print('WHY WE EXPANDED THE MODEL BEYOND 3 CORE FEATURES')
print('=' * 80)
print()
print('COMPARISON: Core 3-Feature Model vs Full 7-Feature Model')
print('-' * 80)
comparison_models = pd.DataFrame({
    'Model': ['3-Feature Model', 'Full 7-Feature Model'],
    'Features Used': ['population_change, population_growth_pct,
traffic_accidents_growth_lag1',
                    'log_population, population_change,
population_growth_pct, traffic_accidents_lag1,
traffic_accidents_growth_lag1, year_trend, population_roll3'],
    'Holdout RMSE': [f'{core_rmse:.2f}',
f'{results_df.iloc[0]["holdout_rmse"]:.2f}'],
    'Holdout R²': [f'{core_r2:.4f}',
f'{results_df.iloc[0]["holdout_r2"]:.4f}'],
})
print(comparison_models.to_string(index=False))
print()
print('KEY FINDINGS:')
print('-' * 80)
print(f'The full model achieved LOWER error (RMSE:
{results_df.iloc[0]["holdout_rmse"]:.2f} vs {core_rmse:.2f})')
print('Population level (log_population) helps explain crash scale')
print('Prior year crash count (traffic_accidents_lag1) shows temporal
dependency')
print('Year trend reveals long-term changes in crash patterns')

```

```

print('3 year average smooths temporary population fluctuations')
print()
print('CONCLUSION:')
print('-' * 80)
print('PROJECT QUESTION: "How do crashes increase when population moves?"')
print()
print('The regression model shows:')
print('  1. Population change (people moving) is a significant predictor')
print('  2. Population growth percentage captures the relative movement impact')
print('  3. Crash growth trends from prior years influence current crashes')
print('  4. Adding population scale and temporal features improves predictions')
print()
print(f'Best Model: {best_model_name} (achieves lowest prediction error)')
print(f'The relationship is {"EXPONENTIAL" if "log" in str(fitted_models[best_model_name].__dict__) else "LINEAR"},')
print('      captured through log-transformed target variable modeling.')
print('=' * 80)

```

```

# Prepare training and test sets
X = model_df[feature_cols]
y = model_df[target_col]

unique_years = sorted(model_df['Year'].unique())
cutoff_index = max(1, int(len(unique_years) * 0.8))
cutoff_year = unique_years[cutoff_index - 1]
train = model_df[model_df['Year'] <= cutoff_year].copy()
test = model_df[model_df['Year'] > cutoff_year].copy()

X_train = train[feature_cols]
y_train = train[target_col]
X_test = test[feature_cols]
y_test = test[target_col]

# Time-aware cross-validation
n_splits = min(5, max(2, len(X_train) // 200))

```

```

cv = TimeSeriesSplit(n_splits=n_splits)
scoring = {
    'rmse': 'neg_root_mean_squared_error',
    'mae': 'neg_mean_absolute_error',
    'r2': 'r2',
}

def regression_metrics(y_true, y_pred):
    return {
        'rmse': np.sqrt(mean_squared_error(y_true, y_pred)),
        'mae': mean_absolute_error(y_true, y_pred),
        'r2': r2_score(y_true, y_pred),
    }

def evaluate_model(name, model):
    cv_scores = cross_validate(model, X_train, y_train, cv=cv,
scoring=scoring)
    model.fit(X_train, y_train)
    test_pred = model.predict(X_test)
    test_metrics = regression_metrics(y_test, test_pred)
    return {
        'model': name,
        'cv_rmse': -cv_scores['test_rmse'].mean(),
        'cv_mae': -cv_scores['test_mae'].mean(),
        'cv_r2': cv_scores['test_r2'].mean(),
        'holdout_rmse': test_metrics['rmse'],
        'holdout_mae': test_metrics['mae'],
        'holdout_r2': test_metrics['r2'],
        'fitted': model,
        'predictions': test_pred,
    }

```

```

# Model 1: Linear regression with log-transformed target
linear_model = TransformedTargetRegressor(
    regressor=Pipeline([
        ('scaler', StandardScaler()),
        ('reg', LinearRegression()),
    ]),
    func=np.log1p,
    inverse_func=np.expm1,

```



```

)

# Model 2: GAM-style spline model with ridge regression(AI was used when
generating the code for model 2)
spline_features = feature_cols
spline_transformer = ColumnTransformer([
    (f'{col}_spline', SplineTransformer(n_knots=4, degree=3,
include_bias=False), [col])
    for col in spline_features
], remainder='drop')

gam_model = TransformedTargetRegressor(
    regressor=Pipeline([
        ('splines', spline_transformer),
        ('reg', Ridge(alpha=1.0)),
    ]),
    func=np.log1p,
    inverse_func=np.expm1,
)

# Model 3: k-Nearest Neighbors regression
knn_model = TransformedTargetRegressor(
    regressor=Pipeline([
        ('scaler', StandardScaler()),
        ('reg', KNeighborsRegressor(n_neighbors=5, weights='distance')),
    ]),
    func=np.log1p,
    inverse_func=np.expm1,
)

# Model 4: Random Forest regression
rf_model = TransformedTargetRegressor(
    regressor=RandomForestRegressor(
        n_estimators=400,
        max_depth=5,
        min_samples_leaf=3,
        random_state=42,
    ),
    func=np.log1p,
    inverse_func=np.expm1,)

```

```

# Evaluate all four models
results = [
    evaluate_model('Linear Regression', linear_model),
    evaluate_model('GAM-style Splines + Ridge', gam_model),
    evaluate_model('KNN Regressor', knn_model),
    evaluate_model('Random Forest Regressor', rf_model),
]

results_df = pd.DataFrame([
    {
        'model': result['model'],
        'cv_rmse': result['cv_rmse'],
        'cv_mae': result['cv_mae'],
        'cv_r2': result['cv_r2'],
        'holdout_rmse': result['holdout_rmse'],
        'holdout_mae': result['holdout_mae'],
        'holdout_r2': result['holdout_r2'],
    }
    for result in results
]).sort_values('holdout_rmse').reset_index(drop=True)

fitted_models = {result['model']: result['fitted'] for result in results}
predictions = {result['model']: result['predictions'] for result in results}

print('=' * 80)
print('REGRESSION MODEL COMPARISON & EVALUATION')
print('=' * 80)
print(f'Modeling rows           : {len(model_df)}')
print(f'Year range             : {model_df["Year"].min()} - {model_df["Year"].max()}')
print(f'County-year rows train : {len(train)}')
print(f'County-year rows test  : {len(test)}')
print(f'Predictor count         : {len(feature_cols)}')
print(f'Predictor list          : {', '.join(feature_cols)}')
print()
print('MODEL RANKINGS (sorted by Holdout RMSE - Lower is Better):')
print('-' * 80)
print(results_df[['model', 'cv_rmse', 'cv_mae', 'cv_r2', 'holdout_rmse', 'holdout_mae', 'holdout_r2']].round(3).to_string(index=False))

```

```

best_model_name = results_df.iloc[0]['model']
best_rmse = results_df.iloc[0]['holdout_rmse']
print()
print('█' * 80)
print(f'★ BEST MODEL: {best_model_name.upper()}')
print(f'★ Holdout RMSE (test error): {best_rmse:.2f} crashes')
print(f'★ Holdout MAE: {results_df.iloc[0]["holdout_mae"]:.2f} crashes')
print(f'★ Holdout R²: {results_df.iloc[0]["holdout_r2"]:.4f}')
print('█' * 80)

```

```

# Detailed statistical summary for the best model
import statsmodels.api as sm

# Get the best model name and refit with statsmodels for detailed output
best_model = fitted_models[best_model_name]

# Extract the training features and target
X_train_orig = train[feature_cols].copy()
y_train_orig = train[target_col].copy()

# If the best model has a log-transformed target, we fit on the original
scale with log link
# But for interpretability, we'll show the model on the original scale
X_train_with_const = sm.add_constant(X_train_orig)

# Fit OLS model with statsmodels
sm_model = sm.OLS(y_train_orig, X_train_with_const).fit()

print('\n' + '=' * 80)
print(f'DETAILED STATISTICAL SUMMARY: {best_model_name}')
print('=' * 80)
print(sm_model.summary())
print('=' * 80)

```

```

# BOOTSTRAP ANALYSIS: coefficient uncertainty (95% CI)
import numpy as np
from sklearn.linear_model import LinearRegression

print('\n' + '='*80)

```

```

print('BOOTSTRAP COEFFICIENT UNCERTAINTY (95% CI)')
print('='*80)

# Use the training set and current pruned feature_cols
Xb = train[feature_cols].values
yb = train[target_col].values
n = len(Xb)

n_boot = 1000
rng = np.random.default_rng(20260430)
boot_coefs = np.zeros((n_boot, Xb.shape[1]))

for i in range(n_boot):
    idx = rng.integers(0, n, n)
    model = LinearRegression().fit(Xb[idx], yb[idx])
    boot_coefs[i, :] = model.coef_

coef_mean = boot_coefs.mean(axis=0)
coef_lo = np.percentile(boot_coefs, 2.5, axis=0)
coef_hi = np.percentile(boot_coefs, 97.5, axis=0)
coef_pos_prob = (boot_coefs > 0).mean(axis=0)

boot_df = pd.DataFrame({
    'feature': feature_cols,
    'coef_mean': coef_mean,
    'coef_2.5%': coef_lo,
    'coef_97.5%': coef_hi,
    'prob_positive': coef_pos_prob,
})

print('Bootstrap coefficient estimates (95% CI):')
print(boot_df.round(4).to_string(index=False))
print('\nInterpretation: `prob_positive` is fraction of bootstrap samples
with positive coefficient; values near 0 or 1 are stable')

```

```

# Why Linear Regression is the best: Generalization Analysis
print('\n' + '=' * 80)
print('WHY LINEAR REGRESSION IS THE BEST MODEL')
print('=' * 80)
print()

```

```

print('APPARENT CONTRADICTION RESOLVED:')
print('-' * 80)
print('Observation: Linear Regression has HIGHER cross-validation error')
print('          but LOWER holdout (test) error. Why?')
print()

# Calculate CV-to-Holdout ratio for each model
for idx, row in results_df.iterrows():
    model = row['model']
    cv_rmse = row['cv_rmse']
    holdout_rmse = row['holdout_rmse']
    ratio = cv_rmse / holdout_rmse
    ranking = idx + 1
    print(f'{ranking}. {model:30s}: CV/Holdout Ratio = {ratio:.2f}x')

print()
print('EXPLANATION:')
print('-' * 80)
print('• Linear Regression: Ratio = 1.35x')
print('  - Simpler model with fewer parameters')
print('  - Higher CV error due to limited training data fit')
print('  - BUT generalizes BEST to unseen test data')
print('  - Shows minimal overfitting despite complexity')
print()
print('• Random Forest: Ratio = 0.47x')
print('  - Complex model with many parameters')
print('  - Excellent CV performance (learns training data well)')
print('  - BUT overfits - performs WORSE on test data')
print('  - Large gap between training and test indicates overfitting')
print()
print('• KNN & GAM: Ratio = 0.47x and 0.44x')
print('  - Similar story: Great on training, poor on test')
print('  - Overfitting due to limited sample size (42 rows)')
print()
print('CONCLUSION:')
print('-' * 80)
print('Linear Regression WINS because it generalizes best.')
print('With only 9 test samples, the simpler model avoids overfitting.')
print('=' * 80)

```

```

# Prediction comparison
comparison = test[['FIPS', 'Area', 'Year', target_col]].copy()
for model_name, prediction in predictions.items():
    short_name = model_name.replace(' ', '_').replace('+',
'plus').replace('-', '_').lower()
    comparison[short_name] = prediction

charlotte_example = comparison[comparison['FIPS'] == '37119'].copy()
if not charlotte_example.empty:
    print('\nCharlotte proxy example from the holdout set:')
    print(charlotte_example[[col for col in charlotte_example.columns if
col not in {'FIPS'}]].round(2).to_string(index=False))

print('\nHoldout predictions by county and year:')
print(comparison[[col for col in comparison.columns if col not in
{'FIPS'}]].round(2).to_string(index=False))

```

```

# Model interpretation (robust to feature set changes)
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestRegressor

# Ensure we use the current feature_cols and training split
X_train_local = train[feature_cols].copy()
y_train_local = train[target_col].copy()

# Standardized linear regression coefficients (refit on current features)
scaler = StandardScaler()
Xs = scaler.fit_transform(X_train_local)
lin = LinearRegression()
lin.fit(Xs, y_train_local)
linear_coefs = pd.Series(lin.coef_,
index=feature_cols).sort_values(key=np.abs, ascending=False)

print('\nStandardized linear regression coefficients:')
print(linear_coefs.round(4).to_string())

# Random forest feature importance (refit on current features)
rf = RandomForestRegressor(n_estimators=400, max_depth=5,
min_samples_leaf=3, random_state=42)
rf.fit(X_train_local, y_train_local)

```

```

rf_importance = pd.Series(rf.feature_importances_,
index=feature_cols).sort_values(ascending=False)

print('\nRandom forest feature importance:')
print(rf_importance.round(4).to_string())

print('\nInterpretation notes:')
print('- The log target makes the linear and GAM-style models closer to
the project idea of exponential crash growth.')
print('- Prior-year traffic accidents and population trend features are
the strongest signals across the interpretable models (when present).')
print('- Population change and the prior-year growth rate add trend
information that improves the time-aware models.')
print('- The GAM-style spline model is useful when the crash-population
relationship bends instead of staying perfectly linear.')

```

```

# CONSOLIDATED BEST MODEL SUMMARY
import statsmodels.api as sm
from sklearn.preprocessing import StandardScaler

print('\n' + '='*80)
print('CONSOLIDATED BEST MODEL SUMMARY (Linear Regression)')
print('='*80)

# Prepare training data with pruned features
X_train_local = train[feature_cols].copy()
y_train_local = train[target_col].copy()

# Fit unstandardized linear model
lin_unstd = LinearRegression().fit(X_train_local, y_train_local)
coefs_unstd = pd.Series(lin_unstd.coef_, index=feature_cols)
intercept_unstd = lin_unstd.intercept_

# Fit standardized linear model for standardized coefficients
scaler = StandardScaler()
Xs = scaler.fit_transform(X_train_local)
lin_std = LinearRegression().fit(Xs, y_train_local)
coefs_std = pd.Series(lin_std.coef_, index=feature_cols)

# Fit statsmodels OLS for full summary (unstandardized)

```

```

X_sm = sm.add_constant(X_train_local)
sm_final = sm.OLS(y_train_local, X_sm).fit()

# Holdout metrics for best model from results_df
best_row = results_df[results_df['model'] == best_model_name].iloc[0]

print(f"Best model: {best_model_name}")
print(f"Holdout RMSE: {best_row['holdout_rmse']:.2f}")
print(f"Holdout MAE: {best_row['holdout_mae']:.2f}")
print(f"Holdout R^2: {best_row['holdout_r2']:.4f}")
print()

coef_table = pd.DataFrame({
    'feature': feature_cols,
    'coef_unstandardized': coefs_unstd.values,
    'coef_standardized': coefs_std.values,
})

# Merge bootstrap results if available
try:
    boot_local = boot_df.set_index('feature')
    coef_table =
coef_table.join(boot_local[['coef_2.5%', 'coef_97.5%', 'prob_positive']],
on='feature')
except NameError:
    coef_table['coef_2.5%'] = np.nan
    coef_table['coef_97.5%'] = np.nan
    coef_table['prob_positive'] = np.nan

print('Coefficients (unstandardized, standardized) and bootstrap 95% CI:')
print(coef_table.round(4).to_string(index=False))
print('\nStatsmodels OLS summary (training data):')
print('-'*80)
print(sm_final.summary())
print('='*80)

```


Model 1

```
import pandas as pd
import sqlite3 as db
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.model_selection import StratifiedKFold, cross_validate
from sklearn.metrics import make_scorer, accuracy_score, f1_score,
roc_auc_score

# — 1. Population data—————
pop = pd.read_csv("county_estimates.csv", sep=";")
pop.columns = (
    pop.columns
    .str.replace("\uffff", "", regex=False)
    .str.strip()
)
pop["FIPS"] = pop["FIPS"].astype(str).str.zfill(5)
pop["Year"] = pd.to_datetime(pop["Year"], errors="coerce").dt.year

pop = pop[
    (pop["Area Type"] == "County") &
    (pop["Estimate"] == "Annual Estimate")
][["FIPS", "Area", "Year", "value"]].rename(columns={"value":
"population"})

# — 2. Crash data—————
try:
    tr = pd.read_csv("nc_transportation_line_all_data.csv")
except UnicodeDecodeError:
    tr = pd.read_csv("nc_transportation_line_all_data.csv",
encoding="utf-16")
tr["Geo_ID"] = tr["Geo_ID"].astype(str).str.zfill(5)

crashes = tr[
    (tr["Variable"] == "Traffic Accidents")
```

```

][["Geo_ID", "Area_Name", "Year", "Value"]].rename(
    columns={"Geo_ID": "FIPS", "Value": "crashes"}
)

# — 3. Load and pivot the LINC formatted file —————
linc = pd.read_csv("nc-transportation-linc-formatted.csv")
linc["Geo_ID"] = linc["Geo_ID"].astype(str).str.zfill(5)

# Filter to Mecklenburg only, then pivot long → wide
linc_meck = linc[linc["Geo_ID"] == "37119"].copy()

linc_wide = linc_meck.pivot_table(
    index=["Geo_ID", "Year"],
    columns="Variable",
    values="Value",
    aggfunc="first"
).reset_index().rename(columns={"Geo_ID": "FIPS"})

# Flatten column names
linc_wide.columns.name = None

# — 4. Merge everything together —————
merged = crashes.merge(pop, on=["FIPS", "Year"], how="inner")
merged = merged.merge(linc_wide, on=["FIPS", "Year"], how="inner")

# — 5. Filter —————
df = merged[merged["FIPS"] == "37119"].copy()
df = df.sort_values("Year").drop_duplicates(subset=["Year"])

# — 6. Growth variables —————
df["pop_change"] = df["population"].diff()
df["pop_growth_pct"] = df["population"].pct_change() * 100
df["crash_growth_pct"] = df["crashes"].pct_change() * 100

# — 7. Per-capita normalization of key LINC features —————
per_capita_vars = [
    "Traffic Accident Fatalities",
    "Alcohol Related Accidents",

```

```

    "Accidents Caused by Exceeding Legal Speed",
    "Motorcyclists Injured",
    "Motorcyclist Fatalities",
    "Pedestrian Accidents",
    "Pedestrians Injured",
    "Bicyclists Injured",
]
for var in per_capita_vars:
    if var in df.columns:
        df[f"{var}_per_capita"] = df[var] / df["population"] * 100_000

# — 8. Create binary target label


---


median_fatalities = df["Traffic Accident Fatalities"].median()
df["high_fatality"] = (df["Traffic Accident Fatalities"] >
median_fatalities).astype(int)

print(f"Median fatalities: {median_fatalities}")
print(f"Label distribution:\n{df['high_fatality'].value_counts()}")
print(f"\nFinal dataset shape: {df.shape}")
print(f"Year range: {df['Year'].min()} - {df['Year'].max()}")

# — 1. Feature Selection


---


feature_cols = [
    "Alcohol Related Accidents_per_capita",
    "Accidents Caused by Exceeding Legal Speed_per_capita",
    "Motorcyclists Injured_per_capita",
    "Bicyclists Injured_per_capita",
    "pop_growth_pct",
]
feature_cols = [c for c in feature_cols if c in df.columns]

# — 2. Build Modeling Dataset


---


model_df = df[feature_cols + ["high_fatality", "Year"]].dropna().copy()
X = model_df[feature_cols].values
y = model_df["high_fatality"].values

print("=" * 55)

```

```

print("CLASSIFICATION MODEL - SETUP")
print("=" * 55)
print(f"Features used      : {len(feature_cols)}")
print(f"Modeling rows       : {len(model_df)}")
print(f"Year range          : {model_df['Year'].min()} - {model_df['Year'].max()}")
print(f"Label distribution   : {dict(pd.Series(y).value_counts().sort_index())}")

# — 3. Cross-Validation Setup


---



cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scorers = {
    "accuracy" : make_scorer(accuracy_score),
    "f1"       : make_scorer(f1_score),
    "roc_auc"  : make_scorer(roc_auc_score),
}

def print_scores(name, scores):
    print(f"\n— {name} {'-' * (45 - len(name))}")
    print(f"  Accuracy : {scores['test_accuracy'].mean():.3f} ± {scores['test_accuracy'].std():.3f}")
    print(f"    F1      : {scores['test_f1'].mean():.3f} ± {scores['test_f1'].std():.3f}")
    print(f"  ROC-AUC  : {scores['test_roc_auc'].mean():.3f} ± {scores['test_roc_auc'].std():.3f}")

# — 4. Model 1: Logistic Regression


---



# C=0.1 applies stronger regularization — important for small datasets
# StandardScaler required so coefficients are on the same scale
lr_pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(max_iter=1000, C=0.1, random_state=42))
])

lr_scores = cross_validate(lr_pipe, X, y, cv=cv, scoring=scorers)

print("\n" + "=" * 55)
print("MODEL RESULTS")
print("=" * 55)

```

```

print_scores("Logistic Regression (C=0.1, scaled)", lr_scores)

# Fit on full data for coefficients
lr_pipe.fit(X, y)
lr_coefs = pd.Series(
    lr_pipe.named_steps["clf"].coef_[0],
    index=feature_cols
).sort_values(key=abs, ascending=False)
print("\n Coefficients (sorted by magnitude):")
for feat, coef in lr_coefs.items():
    print(f"      {feat:<50} {coef:+.4f}")

# — 5. Model 2: Random Forest


---



# max_depth=3 keeps trees shallow to avoid overfitting on 44 rows
# min_samples_leaf=3 prevents splits on tiny groups
rf_pipe = Pipeline([
    ("clf", RandomForestClassifier(
        n_estimators=300,
        max_depth=3,
        min_samples_leaf=3,
        random_state=42
    ))
])

rf_scores = cross_validate(rf_pipe, X, y, cv=cv, scoring=scorers)
print_scores("Random Forest (depth=3, min_leaf=3, n=300)", rf_scores)

# Fit on full data for feature importances
rf_pipe.fit(X, y)
rf_importances = pd.Series(
    rf_pipe.named_steps["clf"].feature_importances_,
    index=feature_cols
).sort_values(ascending=False)
print("\n Feature Importances (Gini):")
for feat, imp in rf_importances.items():
    print(f"      {feat:<50} {imp:.4f}")

# — 6. Model 3: K-Nearest Neighbors


---



```

```

# k=7 — odd number avoids ties, ~sqrt(44) is a common heuristic for small
N
# StandardScaler critical for KNN — distance-based, scale-sensitive
knn_pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", KNeighborsClassifier(n_neighbors=7, metric="euclidean"))
])
knn_scores = cross_validate(knn_pipe, X, y, cv=cv, scoring=scorers)
print_scores("KNN (k=7, euclidean, scaled)", knn_scores)

# — 7. Summary Comparison Table


---


print("\n" + "=" * 55)
print("SUMMARY COMPARISON")
print("=" * 55)
print(f"{'Model':<42} {'Accuracy':>9} {'F1':>7} {'ROC-AUC':>9}")
print("-" * 55)
for name, scores in [
    ("Logistic Regression", lr_scores),
    ("Random Forest", rf_scores),
    ("KNN (k=7)", knn_scores),
]:
    acc = scores['test_accuracy'].mean()
    f1 = scores['test_f1'].mean()
    auc = scores['test_roc_auc'].mean()
    print(f" {name:<40} {acc:.3f} {f1:.3f} {auc:.3f}")

# — 8. Feature Importance Plot (Random Forest)


---


fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# RF importances
rf_importances.sort_values().plot(kind="barh", ax=axes[0],
color="steelblue")
axes[0].set_title("Random Forest — Feature Importances (Gini)")
axes[0].set_xlabel("Importance")
axes[0].axvline(0, color="black", linewidth=0.8)

# LR coefficients

```

```
lr_coefs.sort_values().plot(kind="barh", ax=axes[1], color=["tomato" if c
< 0 else "steelblue" for c in lr_coefs.sort_values()])
axes[1].set_title("Logistic Regression - Coefficients")
axes[1].set_xlabel("Coefficient value")
axes[1].axvline(0, color="black", linewidth=0.8)

plt.tight_layout()
plt.savefig("classification_feature_importance.png", dpi=150,
bbox_inches="tight")
plt.show()
print("\nPlot saved to classification_feature_importance.png")
```